Lab 16 — Hands-on lab: budget, diagnosis, rights and walls

Goal: deploy the API with declared resources, read what the node does with them, trigger an `OOMKilled` and a `Pending`, set a LimitRange and a quota, restart a container and read its previous logs, debug without a shell in the image, give a service account the right to read and nothing else, then close the API's network to everything but the front end.

Prerequisites — Lab 15 done: k3s, the `localhost/api:1.0` image imported into containerd (lab 12).

Files provided (`files/`) - `deployment-api.yaml` — the API with `requests`/`limits` and its Service. - `pod-oom.yaml` — a Pod allocating 100 MB under a 32 Mi limit. - `pod-greedy.yaml` — a Pod requesting 64 Gi. - `limitrange.yaml` — a LimitRange and a ResourceQuota for the namespace. - `rbac-reader.yaml` — ServiceAccount `reader`, Role `pod-reader`, RoleBinding. - `netpol-api.yaml` — NetworkPolicy: only `app=web` Pods reach the API.

Step 1 — Requests, limits and what the node does with them

```
mkdir -p ~/labo-docker/16 && cd ~/labo-docker/16 && cp <lab-path>/files/* .
kubectl create namespace lab16 ; kubectl config set-context --current --namespace=lab16
kubectl apply -f deployment-api.yaml ; kubectl rollout status deployment/api --timeout=60s
kubectl get pods -o custom-
columns=NAME:.metadata.name,QOS:.status.qosClass,REQ:.spec.containers[0].resources.requests.mem
ory,LIM:.spec.containers[0].resources.limits.memory
kubectl describe node | sed -n '/Allocated resources/,/Events/p' | head -9
```

Observe two `Burstable` Pods with `64Mi` / `256Mi`; and in the node, `cpu 400m (2%)` of requests and `memory 268Mi (0%)`: your two Pods (200m, 128Mi) **plus** those of `kube-system`.

```
sleep 30 ; kubectl top nodes ; kubectl top pods
```

Observe `wsl-ubuntu 35m 0% 789Mi 2%` (your values differ) and, for each API Pod, a few millicores and `22Mi` of real memory: well under the 64 Mi request, very far from the limit. If you read `metrics not available yet`, wait another 30 s: metrics-server samples every 15 s.

Explanation. The `Allocated resources` table counts **requests** (what the scheduler reserved); `kubectl top` shows **real consumption**. The two never coincide, and the gap between them is what you tune.

Step 2 — `OOMKilled` and `Pending`

```
kubectl apply -f pod-oom.yaml ; sleep 15 ; kubectl get pod glouton
kubectl describe pod glouton | grep -E 'Reason:|Exit Code|Restart Count'
kubectl get pod glouton -o
jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}' ; echo
```

Observe `glouton 0/1 OOMKilled 1 (14s ago)` (or already `CrashLoopBackOff`); `Reason: OOMKilled` and `Exit Code: 137` **twice** (the current state and the previous one: the relaunched container died the same way), `Restart Count: 1`; and `OOMKilled` in `lastState`.

```
kubectl apply -f pod-greedy.yaml ; sleep 5 ; kubectl get pod trop-gourmand
kubectl describe pod trop-gourmand | grep FailedScheduling | cut -c1-150
kubectl delete pod glouton trop-gourmand --now
```

Observe `trop-gourmand 0/1 Pending` and the event `Warning FailedScheduling … 0/1 nodes are available: 1 Insufficient memory`.

Explanation. The limit is enforced by the cgroup: 100 MB in 32 Mi, the kernel kills, the kubelet restarts. The request is enforced by the scheduler: 64 Gi on a node that does not have them, nobody places, nothing starts.

Step 3 — LimitRange and ResourceQuota

```
kubectl apply -f limitrange.yaml
kubectl run sans-limites --image=docker.io/library/alpine --command -- sleep 3600 ; sleep 2
kubectl get pod sans-limites -o jsonpath='{.spec.containers[0].resources}' ; echo
kubectl describe quota budget | sed -n '/Resource/,$p'
```

Observe a Pod created without `resources` that has some anyway: `{"limits":{"cpu":"250m","memory":"128Mi"},"requests":{"cpu":"50m","memory":"32Mi"}}`; and the quota: `limits.memory 640Mi / 2Gi`, `pods 3 / 10`, `requests.cpu 250m / 2`, `requests.memory 160Mi / 1Gi`.

```
kubectl run enorme --image=docker.io/library/alpine --overrides='{"spec":{"containers":[{"name":"e","image":"docker.io/library/alpine","command":["sleep","1"],"resources":{"limits":{"memory":"4Gi"}}}]}}' 2>&1 | cut -c1-160
kubectl delete pod sans-limites --now
```

Observe `Error from server (Forbidden): pods "enorme" is forbidden: maximum memory usage per Container is 1Gi, but limit is 4Gi`.

Explanation. The LimitRange **completes** Pods without resources (defaults) and **rejects** those exceeding `max`; the quota counts everything living in the namespace. Both act at creation, never afterwards.

Step 4 — Events, restart, `logs --previous`, `debug`

```
kubectl get events --sort-by=.lastTimestamp | tail -5 | cut -c1-120
POD=$(kubectl get pods -l app=api -o jsonpath='{.items[0].metadata.name}')
kubectl exec $POD -- sh -c 'kill 1' ; sleep 5 ; kubectl get pod $POD
kubectl logs $POD --previous | tail -1
```

Observe the namespace's latest events (`Killing`, `Scheduled`, `Pulled`, `Started`…); after `kill 1`, the same Pod with `RESTARTS 1`; and in the **previous** logs, `API arretee proprement.`: the last line of the dead container.

```
kubectl debug $POD -it --image=docker.io/library/alpine --target=api --profile=general -- sh -c
'ps aux | head -3; wget -qO- localhost:8080/actuator/health'
```

Observe after two information lines from `kubectl`: `1 1000 … java -jar /app/api.jar` in `ps` — the API's process, seen from an `alpine` container that is not in the image — then `{"status":"UP"}`.

Explanation. The Pod survived, the container was replaced: `logs` without option shows the new one, `--previous` the old one. `kubectl debug --target` lends the container a shell and tools without putting any in the image.

Step 5 — RBAC: an account that can only read

```
kubectl apply -f rbac-reader.yaml
kubectl auth can-i get pods      --as=system:serviceaccount:lab16:reader
kubectl auth can-i delete pods   --as=system:serviceaccount:lab16:reader
kubectl auth can-i get secrets   --as=system:serviceaccount:lab16:reader
kubectl auth can-i get pods -n default --as=system:serviceaccount:lab16:reader
```

Observe `yes`, `no`, `no`, `no`: reading the Pods here, yes; deleting them, seeing the Secrets, or reading elsewhere, no.

```
kubectl get pods --as=system:serviceaccount:lab16:reader | head -2
kubectl get secrets --as=system:serviceaccount:lab16:reader 2>&1 | cut -c1-140
kubectl auth can-i --list --as=system:serviceaccount:lab16:reader | head -6
```

Observe the Pod list going through; then `Error from server (Forbidden): secrets is forbidden: User "system:serviceaccount:lab16:reader" cannot list resource "secrets"…`; and in `--list`, `pods/log` and `pods` with `[get list watch]`, plus a few generic rights (`selfsubjectreviews`, `/api`…) every authenticated subject holds.

Explanation. The Role says **what** (`pods`, `pods/log`; `get list watch`), the RoleBinding says **who** (`reader`), the namespace bounds it all. `--as` asks the API server to answer "as if": no kubeconfig needed to test.

Step 6 — NetworkPolicy: closing the API

```
kubectl run client --image=docker.io/library/alpine --restart=Never --command -- sleep 600 ;
sleep 5
kubectl exec client -- wget -qO- --timeout=3 http://api:8080/actuator/health ; echo
kubectl apply -f netpol-api.yaml ; sleep 5
kubectl exec client -- wget -qO- --timeout=3 http://api:8080/actuator/health ; echo "exit=$?"
```

Observe `{"status":"UP"}` before the policy; afterwards, `wget: can't connect to remote host (10.43.x.x): Connection refused` and `exit=1`.

```
kubectl label pod client app=web ; sleep 3
kubectl exec client -- wget -qO- --timeout=3 http://api:8080/actuator/health ; echo
kubectl get networkpolicy
```

Observe `{"status":"UP"}` again, without redeploying anything: the label is enough; and `api-only-from-web app=api`.

Explanation. As soon as a policy selects `app=api`, only what it lists gets in. k3s enforces policies (kube-router); the same YAML on a CNI that ignores them would change nothing, and only this test reveals it.

Clean-up

```
kubectl delete namespace lab16
kubectl config set-context --current --namespace=default
```

The namespace takes Deployment, Service, Pods, LimitRange, quota, ServiceAccount, Role, RoleBinding and NetworkPolicy with it. The `api:1.0` image stays in containerd for lab 17.

What you must be able to state now

- `Allocated resources` counts requests; `kubectl top` measures reality; QoS `Burstable` when requests ≠ limits.
- `OOMKilled`, `Exit Code 137`, `RESTARTS` climbing: the memory limit; `Pending` + `Insufficient memory`: the request.
- LimitRange: defaults injected into a Pod without `resources`, refusal beyond `max`; quota: `Used` / `Hard` per namespace.
- `logs --previous` for the dead container, `debug --target` for a shell beside an image without one.
- `auth can-i … --as=system:serviceaccount:ns:name`: yes/no, and a real `Forbidden`.
- NetworkPolicy: `Connection refused` without the label, `UP` with it; enforced by the k3s CNI.